R7021E Advanced Robotics and AI

Lab 1: Introduction to ROS

Group 6

Contributions

Dominic Addo (domadd-2): The ROS 2 package, containing the position controller, the 8-figure trajectory generator, the wall follower, and the launch and parameter files. The experiments in simulation and on the robot as well as the analysis of the rosbags.

Abdalrahman Nasser (abdnas-2): Plotting and video scripts, the figures, and the formatting of the report.

Videos

The videos are a replay of a recorded bag into RViz, showing the robot model, the path so far, the current goal and the laser scan at every instant.

Shared video folder: Google Drive video folder

1. ROS 2

ROS 2 is a framework in which programs, called nodes, exchange messages on named topics. A node publishes on a topic and every subscriber receives the message. On the robot, robot.launch.py starts the driver, which publishes /odom, /scan and /tf and subscribes to /cmd_vel. On the laptop, our node subscribes to /odom and /new_position and publishes /cmd_vel. The two machines find each other so long as both use the same ROS_DOMAIN_ID, which was 32 for turtle2. Turning the robot on is not enough: we also have to start robot.launch.py over SSH, and only then can the laptop move it.

2. Block diagram

Figure 1 shows the two control loops we ran. Both are closed loops at the kinematic level. The control input we send is a forward speed v and a turn rate omega, published as a geometry_msgs/TwistStamped on /cmd_vel at 20 Hz.

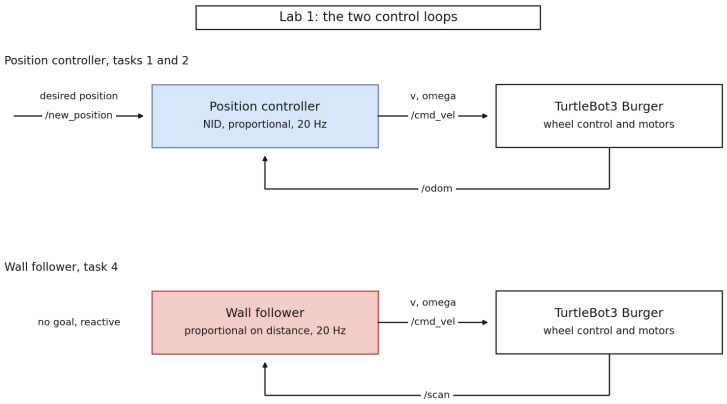


Figure 1: The two control loops. The position controller is driven by a goal on /new_position and closes the loop on odometry. The wall follower has no goal and closes the loop on the laser scan.

There are three levels of control. Level 1 is our code, which turns a goal into v and omega. Level 2 is the wheel speed controller in the OpenCR firmware, which turns v and omega into the speed of each wheel. Level 3 is the motors. We never see levels 2 and 3, which is why we send a velocity rather than a motor command.

The feedback differs between the loops. The position controller closes on the pose from the wheel encoders and IMU, on /odom. The wall follower closes on the raw lidar distances, on /scan, and never uses the pose to steer; it uses odometry only to notice a lap has closed.

3. The robot model, and how the control input is computed

The Burger is a differential drive robot. With wheel speeds v_R and v_L and a wheel separation of 0.160 m, its forward speed and turn rate are

$$v = \frac{(v_R + v_L)}{2} \qquad omega = \frac{(v_R - v_L)}{0.160}$$

and its pose (x, y, theta) in the odometry frame moves as the unicycle model:

$$x_{dot} = v\cos(theta) \quad y_{dot} = v\sin(theta) \quad theta_{dot} = omega$$

The robot cannot move sideways, so its centre cannot be steered straight at a goal. We use the near identity diffeomorphism and control a point P that sits L = 0.10 m ahead of the wheel axle:

$$x_P = x + L\cos(theta) \quad y_P = y + L\sin(theta)$$

P can move in any direction, so a proportional law works on it, with k_p = 0.8:

$$u = k_p(goal - P)$$

The magnitude of u is capped at 0.22 m/s. Inverting the transform gives the two inputs we publish:

$$v = u_x\cos(theta) + u_y\sin(theta)$$

$$omega = \frac{(-u_x\sin(theta) + u_y\cos(theta))}{L}$$

In words: v is the part of u that points along the current heading, and omega is the part that points sideways, divided by L. The node stops when P is within 0.05 m of the goal.

4. Task 1: position control

We sent five goals in turn: (1, 0), (0, 0), (0, 1), (1, 1) and (1, 0). P reached every one of them within the 0.05 m tolerance.

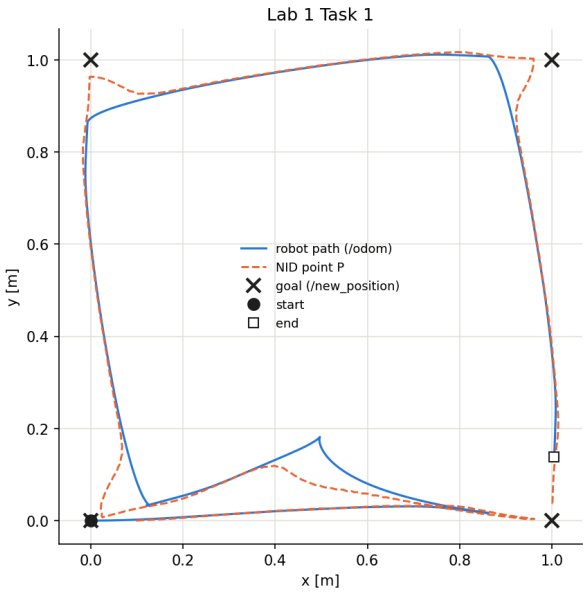


Figure 2: Task 1 on the robot. The path of the robot centre and of the controlled point P, against the goals that were sent.

The centre stopped 0.10 to 0.14 m short of each goal, as expected: the goal is where P stops and P is 0.10 m ahead of the centre. For the first goal, one metre forward, the centre stopped at x = 0.862 m and P at x = 0.961 m. Setting goal_defines to robot_centre shifts the goal forward by L so the centre lands on it instead. The first goal is missing from the bag: it was published once, before the recorder had matched the topic.

Why the robot curves before driving back to a point

When the goal is behind the robot, u points backwards. Its component along the heading is negative, so v is negative and the robot reverses, and its sideways component divided by L = 0.10 m gives a large omega. In the bag, v reached about -0.22 m/s and omega about 2.2 rad/s, which is 0.22 / 0.10. The robot reverses while it swings round, and that is the curve. It drives forward again once P points at the goal, and never turns on the spot first, because the near identity diffeomorphism does not control heading.

5. Task 2: the figure of eight

How the eight is generated

A trajectory node publishes a moving goal on /new_position at 20 Hz. The goal follows a Gerono lemniscate,

$$x = a\,sin\,(s) \quad y = b\,sin\,(2s) \quad s = \frac{2\,pi\,t}{50s}$$

so y completes two oscillations while x completes one, and the curve crosses itself once in the middle. On the robot we used a = 0.5 m and b = 0.25 m, an eight of 1.0 by 0.5 m, to fit the floor space we had, for two laps. The position controller simply chases that goal, so no new control law is needed for this task.

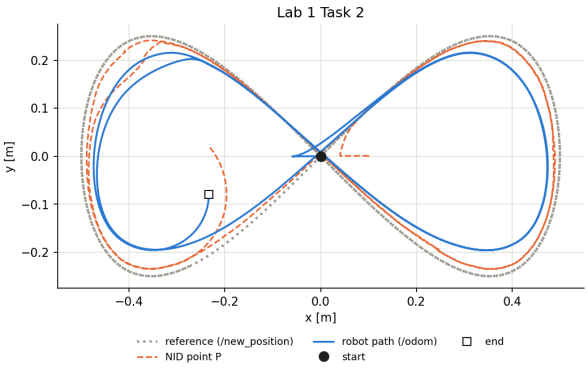


Figure 3: Task 2 on the robot. The path of P against the commanded figure of eight, for two laps.

What we saw

The robot drew the right shape, but late. A proportional controller only acts on an error that already exists, so it follows a moving goal with a delay of about 1 / k_p = 1.25 s. In Gazebo the best fit delay was 1.25 s and on the robot 1.35 s; with that delay removed the remaining error of P is only about 0.014 m. The extra 0.10 s matches the 130 to 270 ms we measured between a command and the robot responding. The mean error of P was 0.083 m, against v / k_p = 0.061 / 0.8 = 0.076 m predicted by the same argument.

Two other things are visible in the bag. The robot first reversed about 5 cm, because P started 0.10 m ahead of the first goal. And odometry stopped reaching the laptop for more than 0.5 s on five occasions while the robot was moving, the longest gap 2.6 s. Each time the controller stopped the robot, because its odometry timeout is 0.5 s, and the tracking error jumped once the stream restarted.

6. Task 4: wall following

How the lidar lets the robot follow a wall

The lidar measures the distance to whatever is around the robot, one beam per degree. The node reads the beams on the right and takes the closest as the distance to the wall. Too far from the wall and it turns towards it, too close and it turns away:

$$omega = k_p(distance - 0.50\,m), \quad k_p = 1.2$$

Two beams 40 degrees apart give the angle of the wall, so the robot straightens up before it drifts. If anything is closer than 0.35 m in front, it stops driving forward and turns on the spot. It stops for good when odometry says it is back where the lap started.

What happened in the lab

We were asked to place a 3D printed maze block of about 0.25 by 0.25 m and follow it to the wall. The robot followed it for a while, but from too far away, then made loops and never got all the way round. It never hit anything, including the teacher when he stepped in front of it, because of the 0.35 m front stop. Our task 4 bag holds only 129 s of the robot standing at its start position, so the motion itself was not recorded.

Why it did that

The follower was tuned for room walls, not for a small block. There are three reasons.

1. The setpoint of 0.50 m is a normal distance from a room wall but a large one from a 25 cm block, so the robot sat far away from it.
2. Each time the block left the right-side beams at a corner, the code treated it as the largest error it handles, 0.5 m, and turned at 1.2 x 0.5 = 0.6 rad/s while driving at 0.15 m/s. That is a circle of 0.25 m radius, while going round the block at 0.50 m needs about 0.63 m, so the robot turned too tightly and made loops.
3. The lap only starts once the two-beam fit sees a straight wall, and a 0.25 m block is too small for both beams to land on. The lap never started, so the robot could never decide that it was back.

We checked this in a desk simulation with our own code, a simulated lidar, the 0.25 m block and the lab day settings. With the block on the right, the robot went round it about 1.9 times in 90 s at a median 0.46 m, and never stopped. Over 96 start poses it went round in 50 percent of runs and closed the loop in none.

The wall follower in simulation

In Gazebo, with the same code and settings as on the lab day, the robot found the wall of the room, followed it round all four sides and stopped within 0.30 m of its lap start after about 15.4 m.

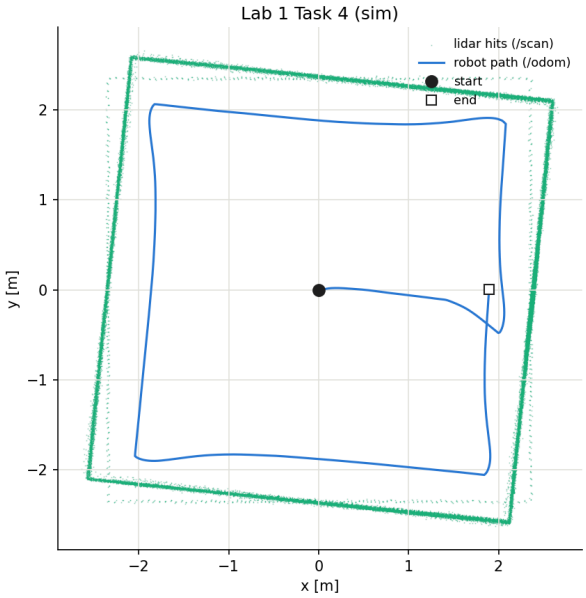


Figure 4: Task 4 in Gazebo. One full lap of the room, stopping where the lap started.

7. Reflections on the experiment

Do you observe any difference between controlling the robot in simulation and controlling the physical robot in real time?

Yes, in four ways that we could measure. The real robot answers a command 130 to 270 ms later and coasts 1 to 2 cm after a stop. Odometry arrived over Wi-Fi with gaps of up to 3.7 s, where in simulation there were none. At the full 0.22 m/s the robot made only about 16 percent of the small turns it was told to make, against 94 percent at the lower speeds used in the figure of eight. And the real lidar returns 0 for a beam that hits nothing, where Gazebo returns infinity; our code ignores both.

How comparable is the robot's actual performance to its performance in simulation?

Close, where the data is clean. The figure of eight lagged 1.35 s on the robot against 1.25 s in Gazebo, and every goal in task 1 was reached in both. The differences are in timing and in turning at full speed. The wall follower is the exception, because the object was the problem and not the robot: it completed a lap of a room in Gazebo and could not complete a lap of the 0.25 m block in the lab.

Where does the physical TurtleBot obtain its odometry information from?

From the robot itself. The driver started by robot.launch.py turns the wheel encoder counts into a pose and publishes it on /odom, with the onboard IMU helping the heading. It is dead reckoning, so it drifts, and it starts at zero wherever the robot happens to be when the driver starts. Nothing corrects it against the outside world.

What changes, adjustments, or additional considerations are required for sim-to-real transfer?

Four were needed before anything moved.

1. use_sim_time was set to false.
2. the right ROS_DOMAIN_ID, which was 32 for turtle2.
3. TwistStamped rather than Twist on /cmd_vel.
4. The lidar ranges of 0 were treated as no echo.

The run itself showed two more: a commanded speed below 0.22 m/s, so that the wheels can still turn when a turn is also demanded, and an odometry timeout loose enough to tolerate the lab Wi-Fi. Our 0.5 s timeout stopped the robot five times in one run.